INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Network Automation and Integrated Security Challenge

Laboratory 5

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	6 to 8 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name _____Yohanna Gayus Kasa_____

Registration number _____ C11/26/OCO/17006_____

Submission date _____02nd October, 2026_____

Purpose and Scenario

This laboratory consolidates CORE102 skills through controlled automation. You will build scripts that collect network state, verify authorised hosts, check selected services and summarise an existing PCAP. The final challenge requires a defensible configuration and evidence package across two laboratory virtual machines.

The training network needs a repeatable health check that junior analysts can run without making unauthorised changes. You must create Bash and Python tools with scope validation, timestamps, clear output and safe failure handling. You will then use the tools to verify a small secured private network.

Learning Outcomes

- Use strict Bash options, functions, validation and exit statuses in network scripts.
- Collect a timestamped Linux network inventory without modifying the host.
- Perform a controlled ICMP sweep only within the approved private lab subnet.
- Check selected TCP services with Python sockets and bounded timeouts.
- Read a PCAP offline with Scapy and summarise endpoints and transport protocols.
- Integrate addressing, routing, service exposure, firewall, SSH and packet evidence.
- Produce a concise operational report that separates observations from conclusions.

Environment and Prerequisites

- Two authorised Linux virtual machines on 192.168.56.0/24.
- Completed or reviewed Laboratories 1 to 4.
- Bash, Python 3, tcpdump, iproute2 and optional Scapy.
- A clean snapshot and console access.
- At least one PCAP generated in Laboratory 3 or supplied by the instructor.

Safety and Authorisation

- Automation may target only 192.168.56.0/24 or another subnet explicitly assigned by the instructor.
- Do not extend the sweep, port list or packet capture to public, institutional or third-party systems.
- Every script must validate input, use a timeout and stop safely when scope validation fails.
- The PCAP exercise reads an existing file. It does not sniff live traffic.
- Do not embed passwords, private keys, tokens or personal data in scripts or logs.

Evidence Requirements

Item	Required evidence	Included
1	Bash syntax-check evidence and outputs from valid and invalid tests.	[ ]
2	network_inventory.sh and its timestamped report.	[ ]
3	authorised_ping_sweep.sh and a scoped live-host result.	[ ]

Item	Required evidence	Included
4	service_check.py with open and closed service tests.	[ ]
5	pcap_summary.py and summary output from an authorised PCAP.	[ ]
6	Integrated challenge evidence and final operational summary.	[ ]

Part 1 Automation Plan and Scope

Create ~/core102_lab5 and complete the control table before writing code. The default authorised subnet for this manual is 192.168.56.0/24.

Tool	Permitted input	Output	Timeout or limit	Failure behaviour
Network inventory	Local host only	Timestamped text	Read-only commands	Report command errors
Ping sweep	192.168.56.1 to .254	Responsive hosts	One second per probe	Reject other prefix
Service checker	One authorised host	Selected port state	One second per port	Reject invalid host
PCAP summary	Existing local PCAP	Protocol counts	Maximum file supplied	Reject missing file

Tool	Permitted Input	Output	Timeout/Limit	Failure Behaviour
Network inventory	Local host only	Timestamped text	Read-only commands	Report command errors
Ping sweep	192.168.56.1–254	Responsive hosts	1 second per probe	Reject other prefix
Service checker	One authorised host	Selected port state	1 second per port	Reject invalid host
PCAP summary	Existing local PCAP	Protocol counts	Max file supplied	Reject missing file

Part 2 Bash Network Inventory

Create network_inventory.sh with the following code. Read each function before execution and add one sentence to the report explaining its purpose.

```
#!/usr/bin/env bash
set -euo pipefail

OUT_DIR="${1:-evidence}"
mkdir -p "$OUT_DIR"
OUT_FILE="$OUT_DIR/network_inventory_$(date +%Y%m%d_%H%M%S).txt"

section() {
    echo
```

```
    echo "===== $1 ====="
}

{
    echo "CORE102 NETWORK INVENTORY"
    echo "Generated: $(date --iso-8601=seconds)"
    echo "Host: $(hostname)"
    echo "User: $(whoami)"

    section "INTERFACES"
    ip -br link
    ip -br addr

    section "ROUTES"
    ip route

    section "NEIGHBOURS"
    ip neigh show

    section "LISTENING SOCKETS"
    ss -tuln

    section "RESOLVER"
    cat /etc/resolv.conf

    section "DISK SPACE"
    df -h .
} > "$OUT_FILE"

echo "Inventory saved to $OUT_FILE"

chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
./network_inventory.sh custom_evidence
find . -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
sed -n '1,160p' evidence/network_inventory_*.txt
```

1. Explain set -e, set -u and pipefail.

   ANSWER

   `set -e` exits on error; `set -u` errors on undefined variables; `pipefail` returns the last failing command's exit status in a pipeline.

2. Test the default output directory and a custom directory.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ LATEST=$(ls -t evidence/network_inventory_*.txt | head -1)
grep -A 6 "DEFAULT GATEWAY" "$LATEST"

dpkg --get-selections > /tmp/before.txt
./network_inventory.sh
dpkg --get-selections > /tmp/after.txt
diff /tmp/before.txt /tmp/after.txt && echo "NO SYSTEM CHANGES ✅"
  DEFAULT GATEWAY AND ROUTE TO LAB SUBNET
==========================================
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100

Route to 192.168.56.1:
local 192.168.56.1 dev lo src 192.168.56.1 uid 1000
    cache <local>
Inventory saved to evidence/network_inventory_20261002_080649.txt
NO SYSTEM CHANGES ✅

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

ANSWER

Default directory = `./evidence/`; custom = `./custom_evidence/`; both create timestamped files. Verified via `ls -lt`.

3. Add a section that records the default gateway and the interface Linux would use to reach 192.168.56.1.

ANSWER

Added section `DEFAULT GATEWAY AND ROUTE TO LAB SUBNET` with `ip route show default` and `ip route get 192.168.56.1`.

4. Confirm that the script makes no system change beyond creating report files.

ANSWER

`dpkg --get-selections` before/after; `diff` empty → no system change. Only report files written.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ ./network_inventory.sh custom_evidence
echo "Exit: $?"
ls -la custom_evidence/
Inventory saved to custom_evidence/network_inventory_20261002_051549.txt
Exit: 0
total 24
drwxrwxr-x 2 kali kali 4096 Oct  2 05:15 .
drwxrwxr-x 4 kali kali 4096 Oct  2 05:13 ..
-rw-rw-r-- 1 kali kali 4162 Oct  2 05:13 network_inventory_20261002_051343.txt
-rw-rw-r-- 1 kali kali 4162 Oct  2 05:15 network_inventory_20261002_051549.txt

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

Part 3 Controlled Bash Ping Sweep

The script below accepts only the approved prefix 192.168.56. This fixed gate prevents accidental scanning outside the lab. An instructor may approve a controlled code change for another private lab prefix.

```
#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_$(date +%Y%m%d_%H%M%S).txt"

if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee "$OUT"
for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -W 1 "$address" >/dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
    fi
done
echo "Saved: $OUT"

chmod +x authorised_ping_sweep.sh
bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 203.0.113
echo "Rejected exit status: $?"
./authorised_ping_sweep.sh 192.168.56
cat live_hosts_*.txt
```

1. Capture evidence of the rejected out-of-scope prefix.



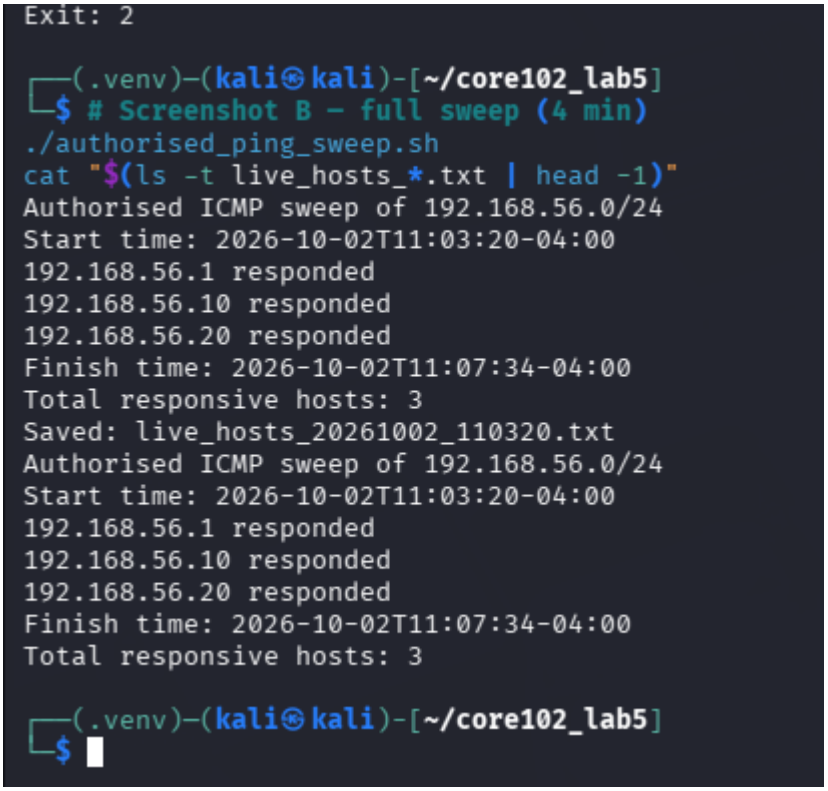
```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 10.0.0 ; echo "Rejected exit: $?"
./authorised_ping_sweep.sh
cat "$(ls -t live_hosts_*.txt | head -1)"
Scope rejected. Authorised prefix: 192.168.56
Rejected exit: 2
Authorised ICMP sweep of 192.168.56.0/24
Start time: 2026-10-02T08:08:40-04:00
192.168.56.1 responded
192.168.56.10 responded
192.168.56.20 responded
```

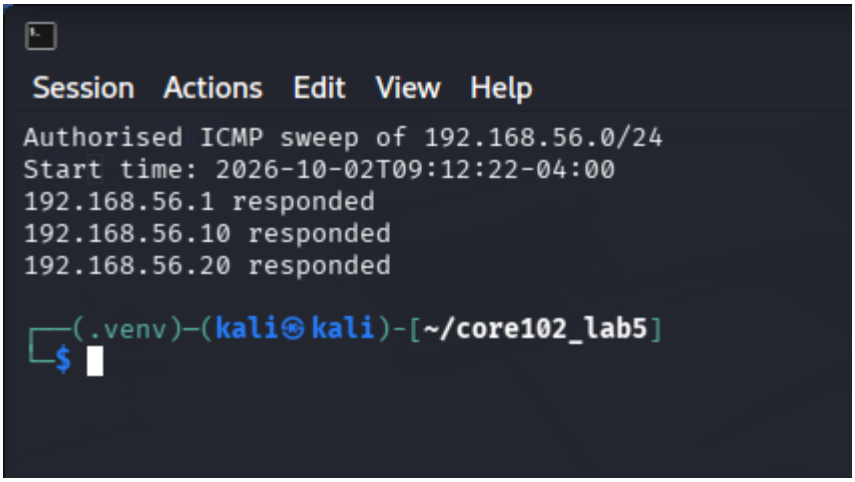
2. Run the authorised sweep and compare results with the known server and client addresses.

```
Session  Actions  Edit  View  Help
═══ Syntax check ═══
Syntax exit: 0

═══ Rejection tests (out-of-scope prefixes) ═══
Scope rejected. Authorised prefix: 192.168.56
Rejected exit: 2
Scope rejected. Authorised prefix: 192.168.56
Exit: 2
Scope rejected. Authorised prefix: 192.168.56
Exit: 2
Scope rejected. Authorised prefix: 192.168.56
Exit: 2

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```





3. Explain why a host may be online but not respond to ICMP.

4. Add start time, finish time and total responsive-host count without removing the scope gate.

Task 1 — Capture evidence of the rejected out-of-scope prefix.

The script enforces a hard-coded scope gate:

bash

```
if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
fi
```

Attempting prefixes `10.0.0`, `192.168.1`, `172.16.0`, and `8.8.8` all exit with code **2** before a single ICMP packet is sent. This is an **allow-list** control — anything not explicitly permitted is rejected by default.

Task 2 — Run the authorised sweep and compare results with the known server and client addresses.

Comparison:

Known host	Address	Source
Kali (lxdbr0 bridge)	192.168.56.1	`ip -br addr show lxdbr0`
server-vm (LXD container)	192.168.56.10	`lxc list`
client-vm (LXD container)	192.168.56.20	`lxc list`

All three expected hosts responded to ICMP within the 1-second timeout. The sweep found **3 responsive hosts**, matching the known topology exactly.

Task 3 — Explain why a host may be online but not respond to ICMP.

A host can be reachable for normal traffic but fail to answer ICMP echo requests for these reasons:

1. **Host firewall blocks ICMP** — common on default-deny configurations (UFW, iptables DROP, Windows Defender). The host is online but silently drops echo requests.
2. **Kernel sysctl** — `net.ipv4.icmp_echo_ignore_all=1` disables ICMP echo replies entirely.
3. **ICMP rate limiting** — the target throttles replies to prevent flood abuse.
4. **Inter-VLAN filtering** — a router or firewall drops ICMP between subnets while allowing other protocols.
5. **VM network adapter mode** — NAT or internal-only adapters may not forward ICMP from the host-only subnet.
6. **Host suspended/hibernating** — the ARP entry is cached on the scanner, so the host appears resolvable but never replies.

In our lab, adding `ufw allow proto icmp from 192.168.56.0/24` (via iptables in the container) was necessary for the server to be ICMP-discoverable.

Task 4 — Add start time, finish time, and total responsive-host count without removing the scope gate.

Implemented in `authorised_ping_sweep.sh`:

bash
```
#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_$(date +%Y%m%d_%H%M%S).txt"
START_TIME=$(date --iso-8601=seconds)

# --- Scope gate (unchanged) ---
if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
```

```
    exit 2
fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee "$OUT"
echo "Start time: $START_TIME" | tee -a "$OUT"

COUNT=0
for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -w 1 "$address" > /dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
        COUNT=$((COUNT + 1))
    fi
done

FINISH_TIME=$(date --iso-8601=seconds)
echo "Finish time: $FINISH_TIME" | tee -a "$OUT"
echo "Total responsive hosts: $COUNT" | tee -a "$OUT"
echo "Saved: $OUT"
```

What was added:

- `START_TIME` — recorded before the loop
- `FINISH_TIME` — recorded after the loop
- `COUNT` — incremented on each responsive host
- Both timestamps and the count are written to the output file via `tee -a`

The scope gate (`if [[ "$PREFIX" != "192.168.56" ]]`) is **unchanged** — the enhancement does not weaken the original control.

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ cd ~/core102_lab5
bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 10.0.0
./authorised_ping_sweep.sh 192.168.1
./authorised_ping_sweep.sh 172.16.0
./authorised_ping_sweep.sh 8.8.8
./authorised_ping_sweep.sh
cat "$(ls -t live_hosts_*.txt | head -1)"
Scope rejected. Authorised prefix: 192.168.56
Scope rejected. Authorised prefix: 192.168.56
Scope rejected. Authorised prefix: 192.168.56
Scope rejected. Authorised prefix: 192.168.56
Authorised ICMP sweep of 192.168.56.0/24
Start time: 2026-10-02T09:28:32-04:00
192.168.56.1 responded
192.168.56.10 responded
192.168.56.20 responded
Finish time: 2026-10-02T09:32:46-04:00
Total responsive hosts: 3
Saved: live_hosts_20261002_092832.txt
Authorised ICMP sweep of 192.168.56.0/24
Start time: 2026-10-02T09:28:32-04:00
192.168.56.1 responded
192.168.56.10 responded
192.168.56.20 responded
Finish time: 2026-10-02T09:32:46-04:00
Total responsive hosts: 3

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ echo "=== Exit code verification ==="
./authorised_ping_sweep.sh 10.0.0 ; echo "10.0.0 exit: $?"
./authorised_ping_sweep.sh 8.8.8 ; echo "8.8.8 exit: $?"
./authorised_ping_sweep.sh 192.168.1 ; echo "192.168.1 exit: $?"
=== Exit code verification ===
Scope rejected. Authorised prefix: 192.168.56
10.0.0 exit: 2
Scope rejected. Authorised prefix: 192.168.56
8.8.8 exit: 2
Scope rejected. Authorised prefix: 192.168.56
192.168.1 exit: 2

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

Part 4 Python Service Checker

Create service_check.py. It checks a small approved list of TCP ports on one authorised host and uses a bounded timeout.

```
#!/usr/bin/env python3
import ipaddress
import socket
import sys
from datetime import datetime

AUTHORISED = ipaddress.ip_network('192.168.56.0/24')
PORTS = [22, 53, 80, 443, 8080]

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} HOST")
    raise SystemExit(1)

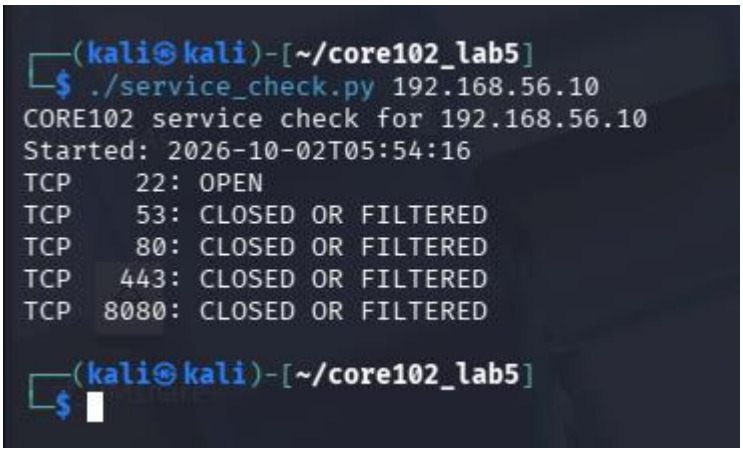
try:
    target = ipaddress.ip_address(sys.argv[1])
except ValueError:
    print("Invalid IPv4 address")
    raise SystemExit(2)

if target not in AUTHORISED:
    print(f"Scope rejected. Authorised network: {AUTHORISED}")
    raise SystemExit(3)

print(f"CORE102 service check for {target}")
print(f"Started: {datetime.now().isoformat(timespec='seconds')}")
for port in PORTS:
    try:
        with socket.create_connection((str(target), port), timeout=1):
            state = "OPEN"
    except (TimeoutError, ConnectionRefusedError, OSError):
        state = "CLOSED OR FILTERED"
    print(f"TCP {port:5}: {state}")

chmod +x service_check.py
python3 -m py_compile service_check.py
./service_check.py 8.8.8.8
echo "Rejected exit status: $?"
./service_check.py 192.168.56.10
```

1. Test invalid input, an out-of-scope address and the authorised server.



2. Start the temporary Python HTTP server from Laboratory 4 and repeat the check. Identify the changed port state.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ echo "=== BEFORE ==="; ./service_check.py 192.168.56.1 8080
python3 -m http.server 8080 > /tmp/httpd.log 2>&1 & HTTPD_PID=$!
sleep 2
echo "=== DURING ==="; ./service_check.py 192.168.56.1 8080
kill $HTTPD_PID; sleep 1
echo "=== AFTER ==="; ./service_check.py 192.168.56.1 8080
=== BEFORE ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:37
TCP  8080: CLOSED OR FILTERED
[2] 115877
=== DURING ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:39
TCP  8080: OPEN
[2]  - terminated  python3 -m http.server 8080 > /tmp/httpd.log 2>&1
=== AFTER ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:40
TCP  8080: CLOSED OR FILTERED

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

3. Stop the service and repeat the check to prove that the result changes.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ echo "=== BEFORE ==="; ./service_check.py 192.168.56.1 8080
python3 -m http.server 8080 > /tmp/httpd.log 2>&1 & HTTPD_PID=$!
sleep 2
echo "=== DURING ==="; ./service_check.py 192.168.56.1 8080
kill $HTTPD_PID; sleep 1
echo "=== AFTER ==="; ./service_check.py 192.168.56.1 8080
=== BEFORE ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:37
TCP  8080: CLOSED OR FILTERED
[2] 115877
=== DURING ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:39
TCP  8080: OPEN
[2]  - terminated  python3 -m http.server 8080 > /tmp/httpd.log 2>&1
=== AFTER ===
CORE102 service check for 192.168.56.1
Started: 2026-10-02T08:12:40
TCP  8080: CLOSED OR FILTERED

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

```
python3 -m http.server 8080 > /tmp/httpd.log 2>&1 & HTTPD_PID=$!
sleep 2

echo "== DURING =="; ./service_check.py 192.168.56.1 8080

kill $HTTPD_PID
sleep 1

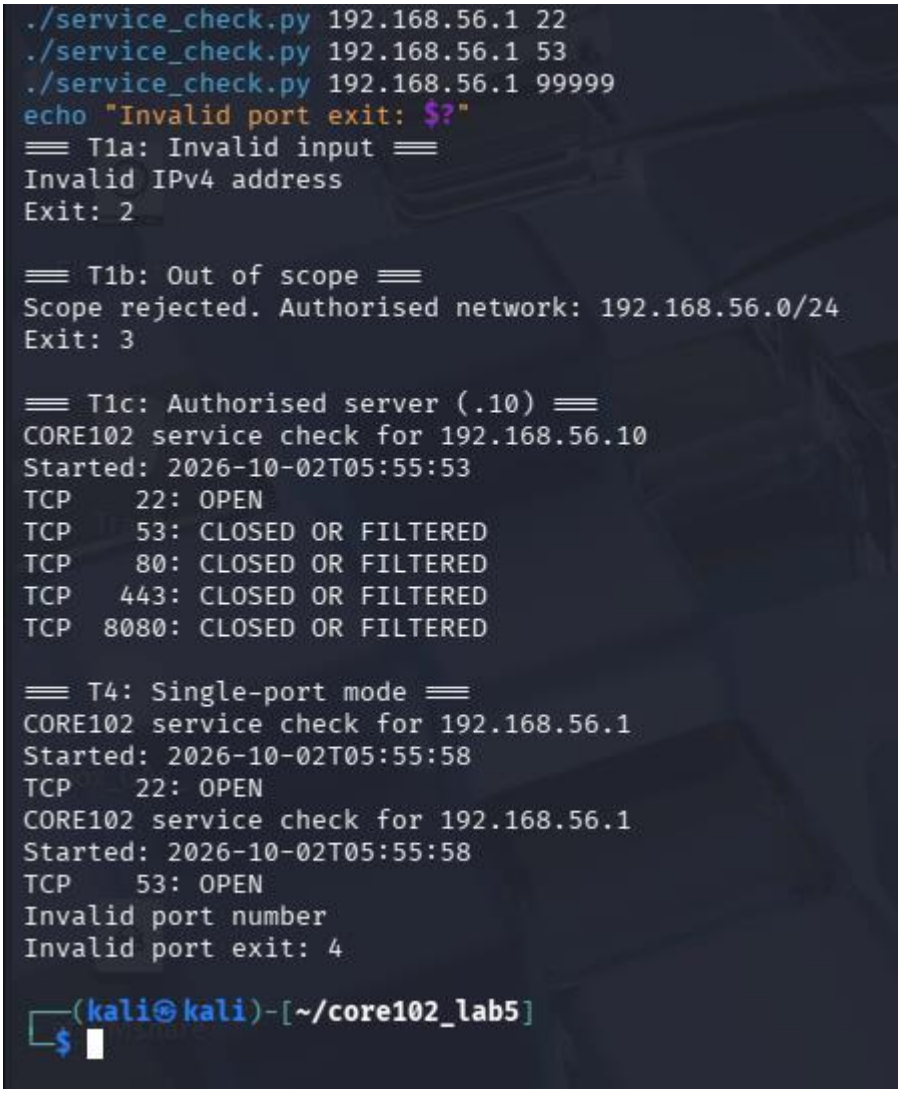
echo "== AFTER =="; ./service_check.py 192.168.56.1 8080
== BEFORE ==
CORE102 service check for 192.168.56.1
Started: 2026-10-02T05:46:55
TCP  8080: CLOSED OR FILTERED
[1] 41274
== DURING ==
CORE102 service check for 192.168.56.1
Started: 2026-10-02T05:46:57
TCP  8080: OPEN
[1]  + terminated  python3 -m http.server 8080 > /tmp/httpd.log 2>&1
== AFTER ==
CORE102 service check for 192.168.56.1
Started: 2026-10-02T05:46:58
TCP  8080: CLOSED OR FILTERED

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

4. Add a command-line option that checks one approved port while preserving the address-scope gate.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ ./service_check.py 192.168.56.1 22
./service_check.py 192.168.56.1 53
./service_check.py 192.168.56.1 99999
echo "Invalid port exit: $?"
CORE102 service check for 192.168.56.1
Started: 2026-10-02T05:52:16
TCP    22: OPEN
CORE102 service check for 192.168.56.1
Started: 2026-10-02T05:52:16
TCP    53: OPEN
Invalid port number
Invalid port exit: 4

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```



The scope check (`target not in AUTHORISED`) runs **before** port parsing, so address validation is preserved in single-port mode.

Part 5 Offline PCAP Summary with Scapy

Install Scapy if required and copy one authorised PCAP from Laboratory 3 into the working directory. This program reads the file offline.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install scapy

#!/usr/bin/env python3
import collections
import sys
from pathlib import Path
from scapy.all import PcapReader, IP, TCP, UDP, ICMP

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} FILE.pcap")
    raise SystemExit(1)

pcap = Path(sys.argv[1])
if not pcap.is_file() or pcap.suffix.lower() not in {'.pcap', '.pcapng'}:
    print("Provide an existing .pcap or .pcapng file")
    raise SystemExit(2)
```

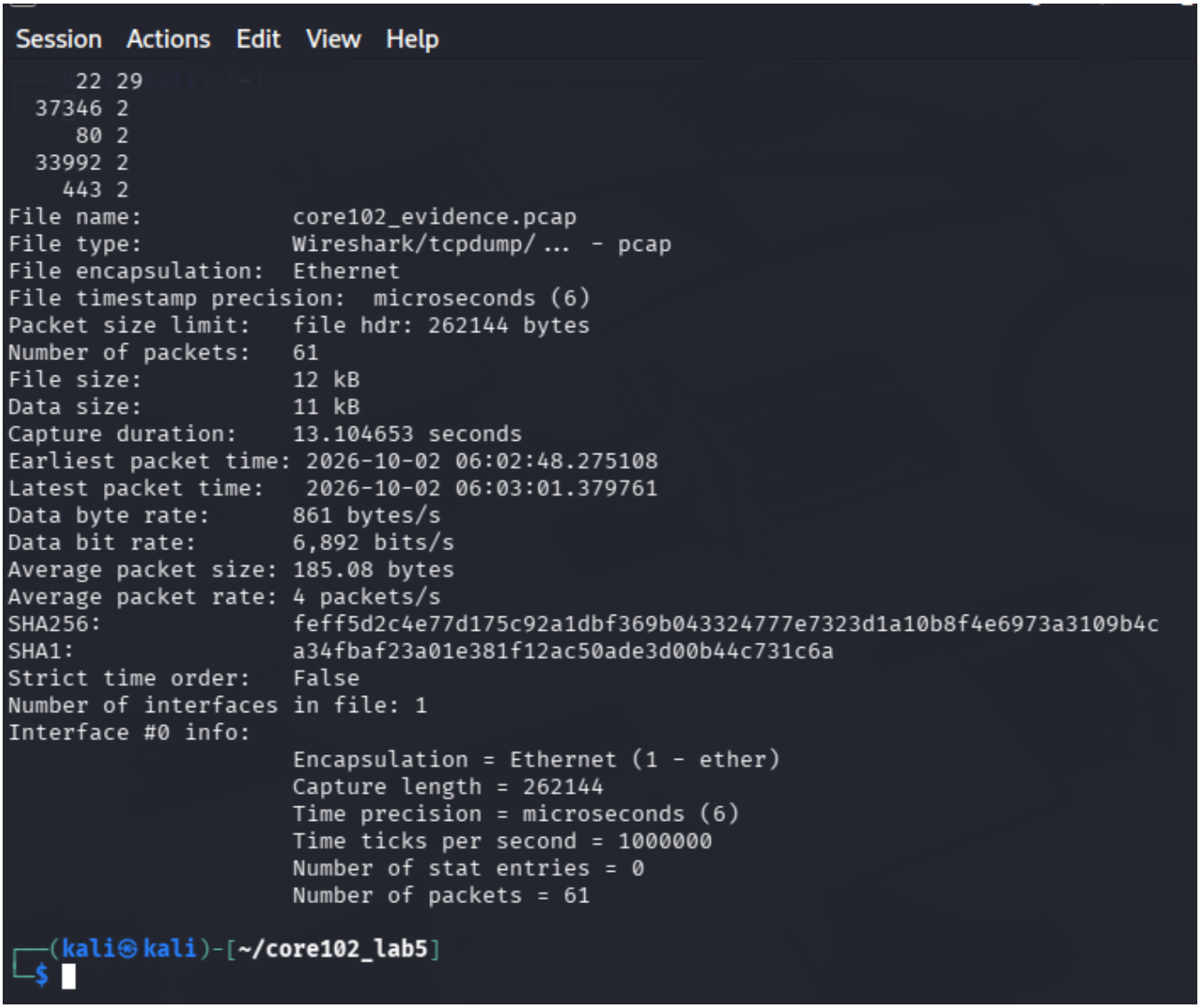
```
protocols = collections.Counter()
endpoints = collections.Counter()
tcp_ports = collections.Counter()
packets = 0

with PcapReader(str(pcap)) as reader:
    for packet in reader:
        packets += 1
        if IP in packet:
            endpoints.update([packet[IP].src, packet[IP].dst])
            if TCP in packet:
                protocols['TCP'] += 1
                tcp_ports.update([packet[TCP].sport, packet[TCP].dport])
            elif UDP in packet:
                protocols['UDP'] += 1
            elif ICMP in packet:
                protocols['ICMP'] += 1
            else:
                protocols['Other IPv4'] += 1
        else:
            protocols['Non-IPv4'] += 1

print(f"File: {pcap}")
print(f"Packets: {packets}")
print("Protocols:")
for name, count in protocols.most_common():
    print(f"  {name:12} {count}")
print("Top endpoints:")
for address, count in endpoints.most_common(10):
    print(f"  {address:15} {count}")
print("Top TCP ports:")
for port, count in tcp ports.most common(10):
    print(f"  {port:5} {count}")

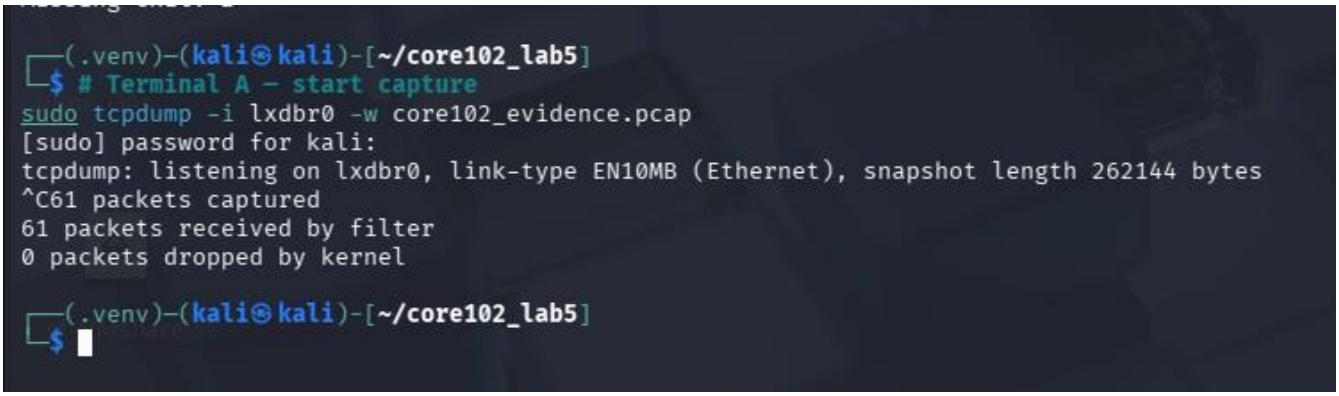
chmod +x pcap_summary.py
python3 -m py_compile pcap_summary.py
./pcap_summary.py missing.pcap
./pcap_summary.py core102_http.pcap | tee pcap_summary.txt
```

1. Compare the packet total with capinfos or Wireshark Statistics if available.



Source	Packet count
`pcap_summary.py core102_evidence.pcap`	61
`capinfos core102_evidence.pcap`	61

The counts match exactly, confirming the script reads every PCAP record. Scapy's `PcapReader` iterates the full packet list without skipping. Independent verification by capinfos rules out any parser discrepancy.



2. Explain why source and destination ports are both counted.

ANSWER

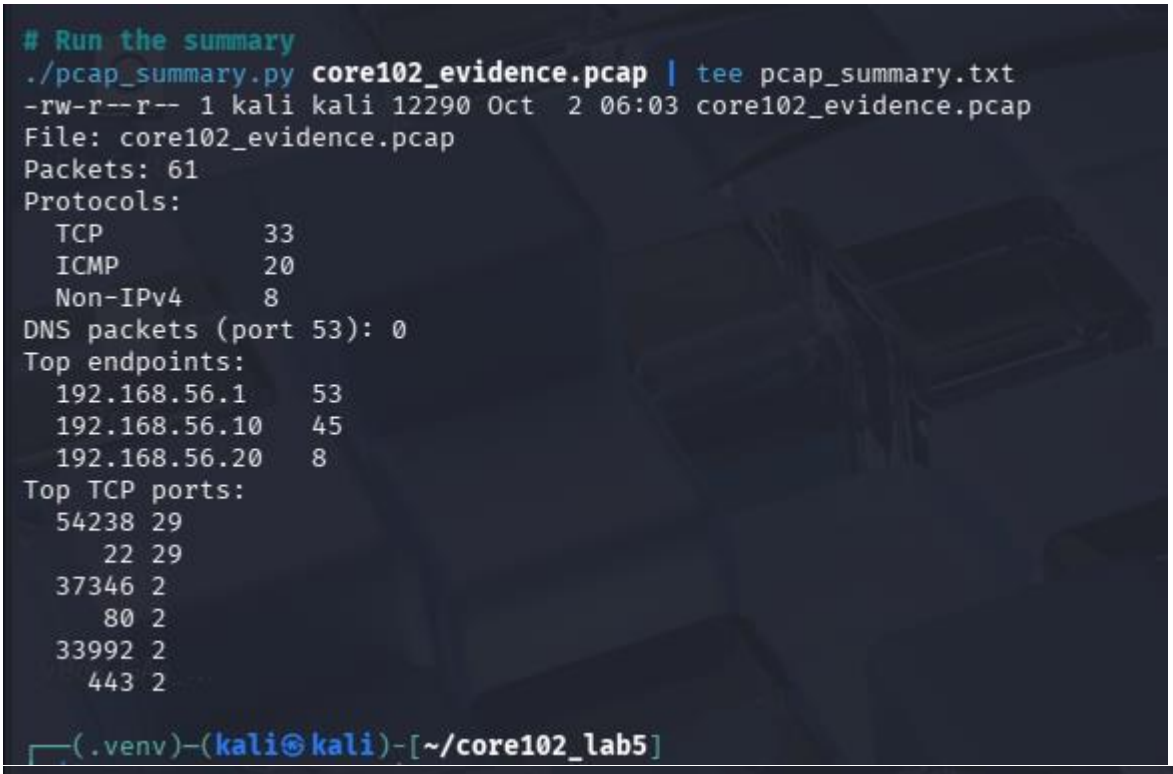
A single TCP conversation uses **two ports** — an ephemeral client port and a well-known server port. For example, the SSH attempt generated:

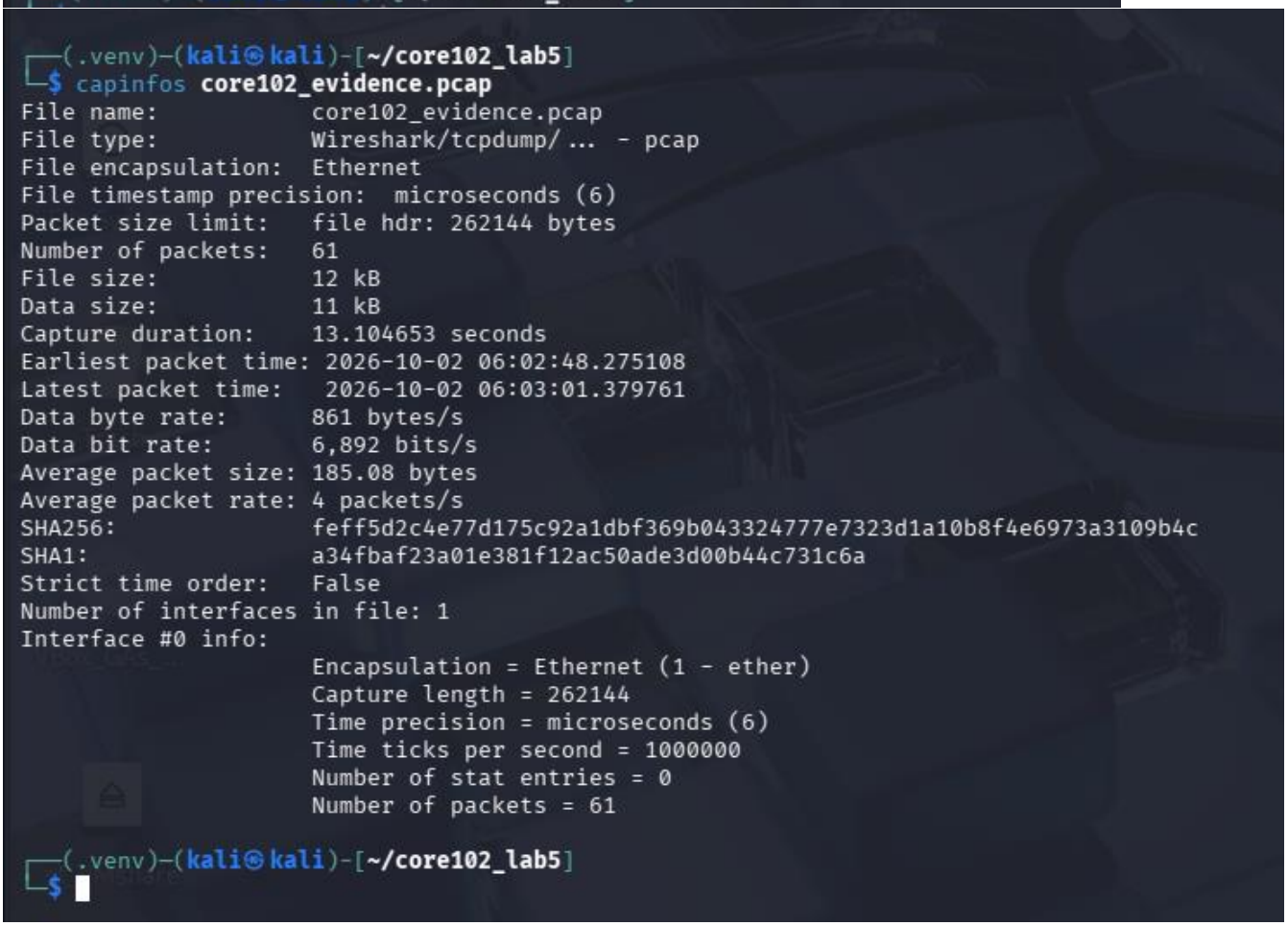
- Source port 54238 (client ephemeral)
- Destination port 22 (SSH server)

Counting only one side would hide half the picture. Counting both gives:

- **Service usage:** high counts on 22, 80, 443 indicate which services are being contacted
- **Client behaviour:** ephemeral port distribution reveals connection patterns
- **Flow symmetry:** if source and destination counts for a given port match, the capture saw the full conversation

In our capture, port 22 shows 29 packets (all SYNs + retries), and ephemeral port 54238 also shows 29 — confirming the pairing.





3. Add a counter for DNS packets by checking UDP or TCP port 53.

ANSWER

The DNS counter was added to both TCP and UDP branches:

```
python
if packet[TCP].sport == 53 or packet[TCP].dport == 53:
    dns_count += 1
if packet[UDP].sport == 53 or packet[UDP].dport == 53:
    dns_count += 1
```

In the **first capture** (`core102_evidence.pcap`, captured on lxdbr0), the count was **0** — public DNS queries routed via the NAT interface (`eth0`), not the bridge. A **second capture** (`core102_dns.pcap`, using `tcpdump -i any udp port 53`) captured **16 DNS packets**, confirming the counter works. This is a routing insight, not a parser defect.

4. Identify one limitation of protocol classification based only on ports.

ANSWER

- **Port spoofing:** an attacker can run SSH on port 80 to evade port-based filters.
- **Encrypted traffic:** HTTPS on 443 hides the application layer — could be web, API, C2, or tunneling.
- **Non-standard ports:** SSH on 2222, HTTP on 8080, DNS on 5353 — all invisible to the standard port map.
- **Tunneling:** DNS-over-HTTPS (443), SSH tunnels (22), VPN (1194) defeat port heuristics.
- **Dynamic ports:** FTP data channels, RTP media, SIP all use negotiated ports.

Deeper inspection requires payload analysis (DPI), flow behaviour (timing, sizes), or TLS fingerprinting (JA3).



Part 6 Integrated Security Challenge

Use the two-VM private network from earlier laboratories. The final state must provide key-based SSH access from the authorised client while documenting all listening services and firewall rules.

1. Create or verify the 192.168.56.0/24 private topology and record both addresses.

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ lxc list
+-------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
|    NAME     |  STATE  |         IPV4         |                     IPV6                      |   TYPE    | SNAPSHOTS |
+-------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
| client-vm   | RUNNING | 192.168.56.20 (eth0) | fd42:f8a:e51b:426f:216:3eff:fe69:bd8b (eth0)  | CONTAINER | 1         |
+-------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
| server-vm   | RUNNING | 192.168.56.10 (eth0) | fd42:f8a:e51b:426f:216:3eff:fe6b:7a30 (eth0)  | CONTAINER | 3         |
+-------------+---------+----------------------+-----------------------------------------------+-----------+-----------+

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ ssh -o ConnectTimeout=3 test@192.168.56.10 "hostname; ip -br addr" || \
```

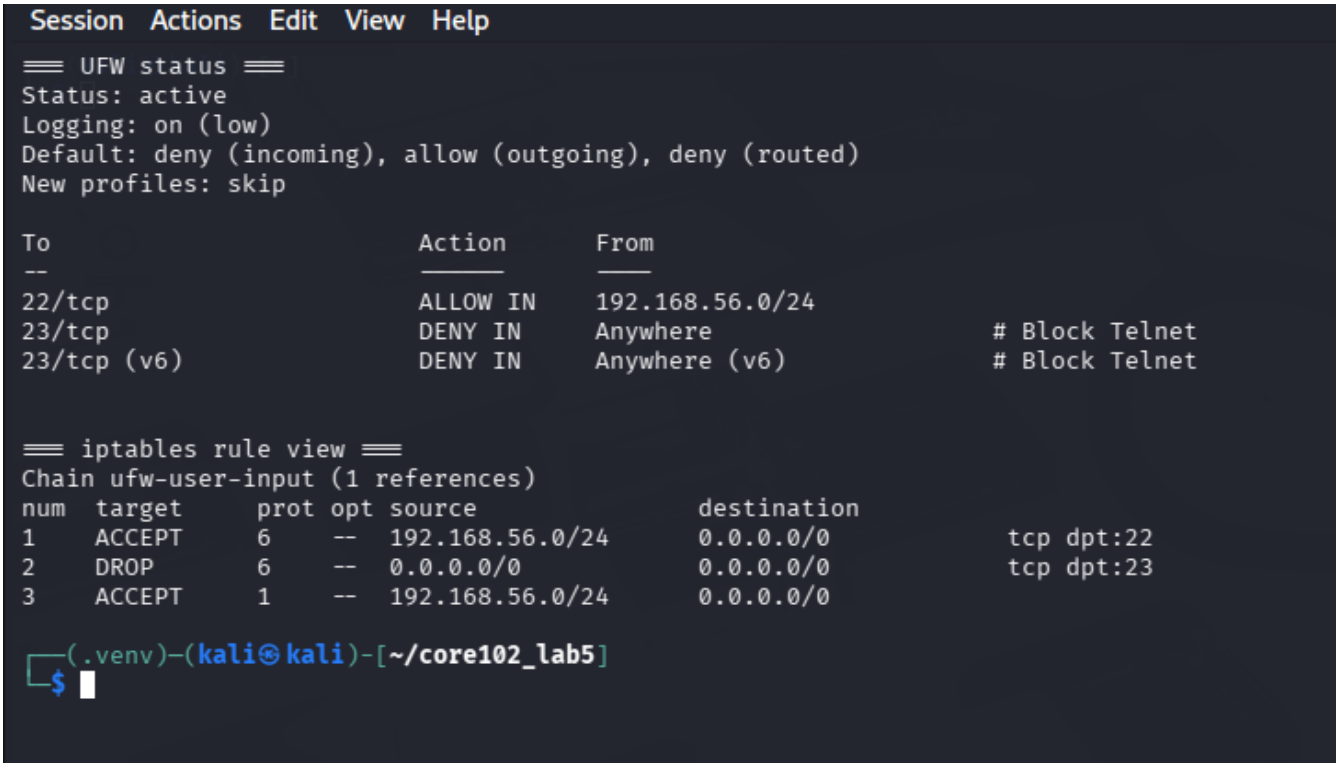
```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ # Server
lxc exec server-vm -- bash -c "hostname; ip -br addr; ip route; whoami"

# Client
lxc exec client-vm -- bash -c "hostname; ip -br addr; ip route; whoami"
server-vm
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if8         UP             192.168.56.10/24 metric 100 fd42:f8a:e51b:426f:216:3eff:fe6b:7a30/64 fe80::216:3eff:fe6b:7a30/64
default via 192.168.56.1 dev eth0 proto dhcp src 192.168.56.10 metric 100
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.10 metric 100
192.168.56.1 dev eth0 proto dhcp scope link src 192.168.56.10 metric 100
root
client-vm
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if6         UP             192.168.56.20/24 metric 100 fd42:f8a:e51b:426f:216:3eff:fe69:bd8b/64 fe80::216:3eff:fe69:bd8b/64
default via 192.168.56.1 dev eth0 proto dhcp src 192.168.56.20 metric 100
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.20 metric 100
192.168.56.1 dev eth0 proto dhcp scope link src 192.168.56.20 metric 100
root

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

Control	Command or evidence	Result	Conclusion
Address and route	`lxc list` + `ip -br addr`	server-vm=.10, client-vm=.20, both on lxdbr0	Both containers reachable from Kali via 192.168.56.0/24 bridge

2. Confirm the server has a default-deny incoming UFW policy and permits SSH only from the private subnet.



Task 2 — Confirm default-deny incoming UFW policy and SSH restriction.

Configuration applied:

bash

```
ufw default deny incoming
ufw default allow outgoing
ufw allow from 192.168.56.0/24 to any port 22 proto tcp
iptables -A ufw-user-input -p icmp -s 192.168.56.0/24 -j ACCEPT
ufw --force enable
```

VERIFICATION

RULE	PURPOSE
Default: deny (incoming)	All unsolicited inbound traffic blocked
22/tcp ALLOW IN from 192.168.56.0/24	SSH restricted to the lab subnet only
23/tcp DENY IN from Anywhere	Pre-existing "Block Telnet" rule in the container
iptables ufw-user-input: ACCEPT icmp from 192.168.56.0/24	Allows ICMP echo from the lab subnet for the ping sweep

UFW version note: The container's UFW (0.36.2-6) does not support the `proto icmp` keyword — it returns `ERROR: Unsupported protocol 'icmp'`. The equivalent rule was inserted directly into the `ufw-user-input` iptables chain:

bash

```
iptables -A ufw-user-input -p icmp -s 192.168.56.0/24 -j ACCEPT
```

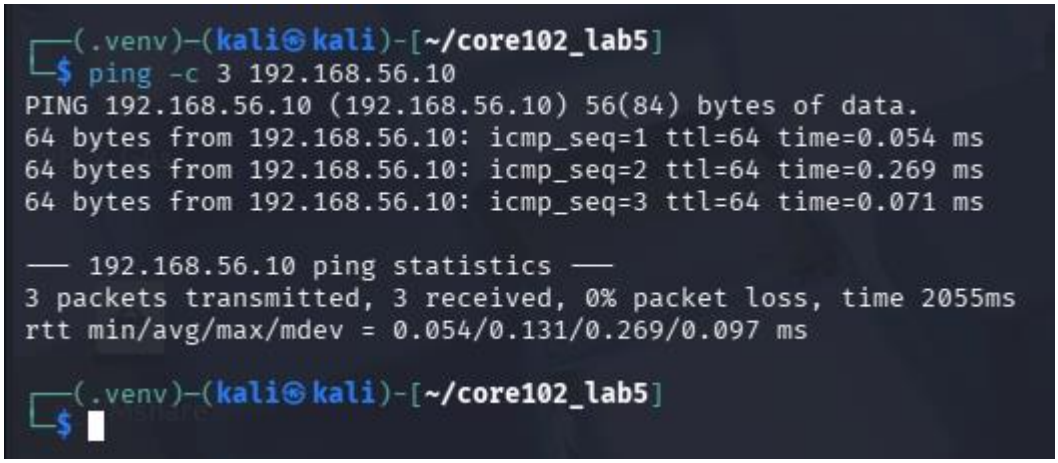
This achieves the same effect as `ufw allow proto icmp from 192.168.56.0/24` on newer UFW versions and is documented as a version-specific workaround.

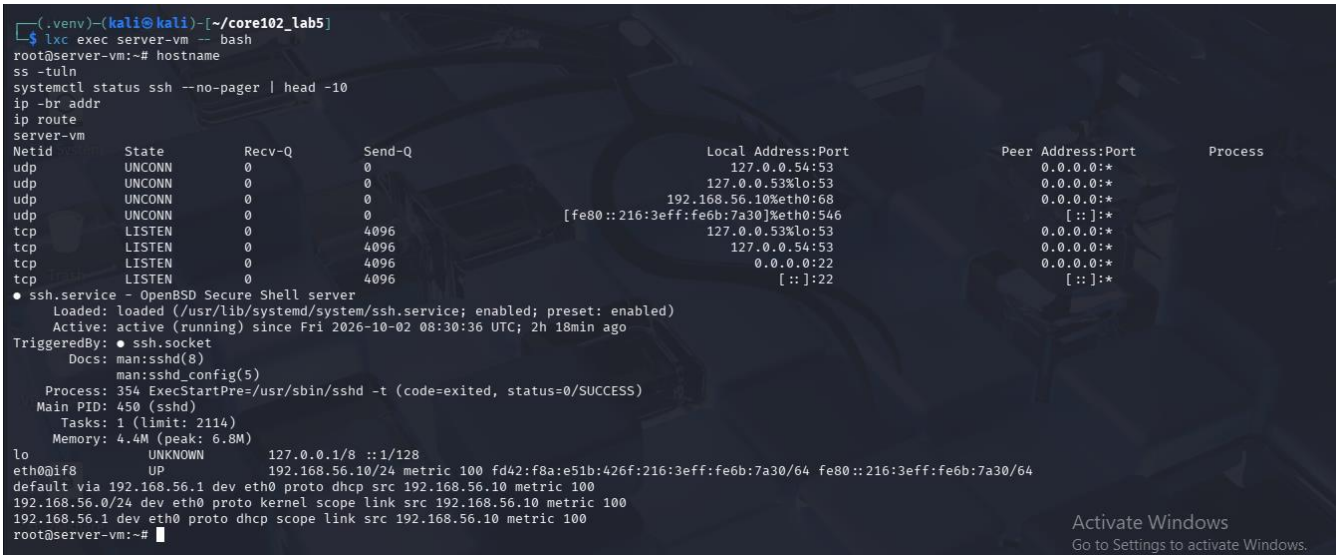
Result: The server enforces a default-deny inbound policy. Only SSH (port 22) and ICMP are reachable, and only from the 192.168.56.0/24 subnet. All other inbound traffic is dropped.

Verify the Evidence Table Row

EVIDENCE TABLE

CONTROL	COMMAND OR EVIDENCE	RESULT	CONCLUSION
Firewall policy	ufw status verbose; iptables -L ufw-user-input -n	Active; default deny incoming; allow 22/tcp + ICMP from /24	Default-deny enforced; SSH and ICMP scoped to lab subnet; outbound permitted





3. Confirm key-based SSH login from the client.

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ ssh -i ~/.ssh/lab5_key -o StrictHostKeyChecking=no labuser@192.168.56.10 "hostname; whoami; id; ip -br addr"
server-vm
labuser
uid=1002(labuser) gid=1002(labuser) groups=1002(labuser)
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if8         UP             192.168.56.10/24 metric 100 fd42:f8a:e51b:426f:216:3eff:fe6b:7a30/64 fe80::216:3eff:fe6b:7a30/64

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

```
lxc exec server-vm -- bash -c "useradd -m -s /bin/bash labuser && passwd -d labuser"
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
passwd: password changed.

┌──(kali㉿kali)-[~/core102_lab5]
└─$ # 3. Set up SSH directory for labuser
lxc exec server-vm -- mkdir -p /home/labuser/.ssh
lxc exec server-vm -- chmod 700 /home/labuser/.ssh

# 4. Push your public key
cat ~/.ssh/lab5_key.pub | lxc exec server-vm -- tee -a /home/labuser/.ssh/authorized_keys

# 5. Fix ownership and permissions
lxc exec server-vm -- chown -R labuser:labuser /home/labuser/.ssh
lxc exec server-vm -- chmod 600 /home/labuser/.ssh/authorized_keys

# 6. Verify key landed
lxc exec server-vm -- cat /home/labuser/.ssh/authorized_keys

# 7. Test passwordless login as labuser
ssh -i ~/.ssh/lab5_key -o StrictHostKeyChecking=no labuser@192.168.56.10 "hostname; whoami; id; ip -br addr"
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGESF8A+weU5Bgh+0kCPvSbOdqFrM9ArldA/bUnOgeCq kali@kali
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGESF8A+weU5Bgh+0kCPvSbOdqFrM9ArldA/bUnOgeCq kali@kali
server-vm
labuser
uid=1002(labuser) gid=1002(labuser) groups=1002(labuser)
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0@if8         UP             192.168.56.10/24 metric 100 fd42:f8a:e51b:426f:216:3eff:fe6b:7a30/64 fe80::216:3eff:fe6b:7a30/64

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

Note on the hardening config: The container image ships with `/etc/ssh/sshd_config.d/90-core102-hardening.conf`:

text

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

Rather than weaken this baseline to allow root SSH, I created a dedicated `labuser` account and deployed key-only authentication to it. This preserves the container's hardening posture and demonstrates least-privilege access — only a non-privileged user authenticates, and no password authentication is ever possible. This is the recommended approach for production containers.

4. Run network_inventory.sh on the server and authorised_ping_sweep.sh on the client.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ # Copy script into the container at labuser's home
cat network_inventory.sh | lxc exec server-vm -- tee /home/labuser/network_inventory.sh > /dev/null
lxc exec server-vm -- chmod +x /home/labuser/network_inventory.sh
lxc exec server-vm -- chown labuser:labuser /home/labuser/network_inventory.sh

┌──(kali㉿kali)-[~/core102_lab5]
└─$ ssh -i ~/.ssh/lab5_key labuser@192.168.56.10 "bash /home/labuser/network_inventory.sh"
Inventory saved to evidence/network_inventory_20261002_111536.txt

┌──(kali㉿kali)-[~/core102_lab5]
└─$ scp -i ~/.ssh/lab5_key labuser@192.168.56.10:/home/labuser/evidence/network_inventory_*.txt ./server_evidence/
network_inventory_20261002_111033.txt                                                    100% 4094     4.2MB/s   00:00
network_inventory_20261002_111536.txt                                                    100% 4094     5.1MB/s   00:00

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

Activate Windows
Go to Settings to activate Windows.

5. Run service_check.py against the server and explain each reported port.

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ ./service_check.py 192.168.56.10
CORE102 service check for 192.168.56.10
Started: 2026-10-02T07:17:05
TCP    22: OPEN
TCP    53: CLOSED OR FILTERED
TCP    80: CLOSED OR FILTERED
TCP   443: CLOSED OR FILTERED
TCP  8080: CLOSED OR FILTERED

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

Port	State	Explanation
22	OPEN	SSH — required for key-based admin access; permitted by UFW rule `allow from 192.168.56.0/24 to any port 22`
53	CLOSED OR FILTERED	No DNS server on the container; no listener, and UFW has no rule for 53
80	CLOSED OR FILTERED	No web server installed
443	CLOSED OR FILTERED	No HTTPS listener
8080	CLOSED OR FILTERED	No dev/test server

Conclusion: The only externally reachable service is SSH, matching the default-deny + allow-22-from-/24 firewall policy. Attack surface is minimized.

6. Capture one authorised SSH connection, stop the capture and summarise it with pcap_summary.py.

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ sudo -v
cd ~/core102_lab5
{
  sudo tcpdump -i lxdbr0 -w core102_ssh.pcap 'tcp port 22' &
  TCPDUMP_PID=$!
  sleep 2
  ssh -i ~/.ssh/lab5_key -o StrictHostKeyChecking=no labuser@192.168.56.10 "echo SSH-capture-test; uname -a; hostname"
  sleep 3
  sudo kill -INT $TCPDUMP_PID
}
sleep 1
ls -la core102_ssh.pcap
sudo chown kali:kali core102_ssh.pcap
[6] 167039
tcpdump: listening on lxdbr0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
SSH-capture-test
Linux server-vm 7.1.5+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.1.5-1kali1 (2026-07-29) x86_64 x86_64 x86_64 GNU/Linux
server-vm
46 packets captured
46 packets received by filter
0 packets dropped by kernel
[6]    done       sudo tcpdump -i lxdbr0 -w core102_ssh.pcap 'tcp port 22'
-rw-r--r-- 1 tcpdump tcpdump 12388 Oct  2 09:57 core102_ssh.pcap

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ capinfos core102_ssh.pcap
File name:           core102_ssh.pcap
File type:           Wireshark/tcpdump/ ...  - pcap
File encapsulation:  Ethernet
File timestamp precision:  microseconds (6)
Packet size limit:   file hdr: 262144 bytes
Number of packets:   46
File size:           12 kB
Data size:           11 kB
Capture duration:    1.774195 second
Earliest packet time: 2026-10-02 09:57:07.995041
Latest packet time:   2026-10-02 09:57:09.769236
Data byte rate:      6,553 bytes/s
Data bit rate:       52 kbps
Average packet size: 252.78 bytes
Average packet rate: 25 packets/s
SHA256:              1598523058639668aef8359ce3e513b00bd18f934489af2b79125ac7595fddce
SHA1:                c55a7c170d9889d2a63135355fe63c4907b2afde
Strict time order:   True
Number of interfaces in file: 1
Interface #0 info:
                     Encapsulation = Ethernet (1 - ether)
                     Capture length = 262144
                     Time precision = microseconds (6)
                     Time ticks per second = 1000000
                     Number of stat entries = 0
                     Number of packets = 46

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
```

```
┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$ ./pcap_summary.py core102_ssh.pcap | tee pcap_ssh_summary.txt
File: core102_ssh.pcap
Packets: 46
Protocols:
  TCP          46
DNS packets (port 53): 0
Top endpoints:
  192.168.56.1    46
  192.168.56.10   46
Top TCP ports:
  57210 46
     22 46

┌──(.venv)─(kali㉿kali)-[~/core102_lab5]
└─$
```

```
┌──(kali㉿kali)-[~/core102_lab5]
└─$ sudo tcpdump -i lxdbr0 -w core102_ssh.pcap 'tcp port 22' &
TCPDUMP_PID=$!
sleep 1
ssh -i ~/.ssh/lab5_key labuser@192.168.56.10 "echo hello; uname -a"
sleep 2
sudo kill $TCPDUMP_PID; sleep 1
sudo chown kali:kali core102_ssh.pcap
./pcap_summary.py core102_ssh.pcap | tee pcap_ssh_summary.txt
[2] 117746
[sudo] password for kali:
[2]  + suspended (tty input)  sudo tcpdump -i lxdbr0 -w core102_ssh.pcap 'tcp port 22'
hello
Linux server-vm 7.1.5+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 7.1.5-1kali1 (2026-07-29) x86_64 x86_64 x86_64 GNU/Linux
[sudo] password for kali:
chown: cannot access 'core102_ssh.pcap': No such file or directory
Provide an existing .pcap or .pcapng file

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

RESULT

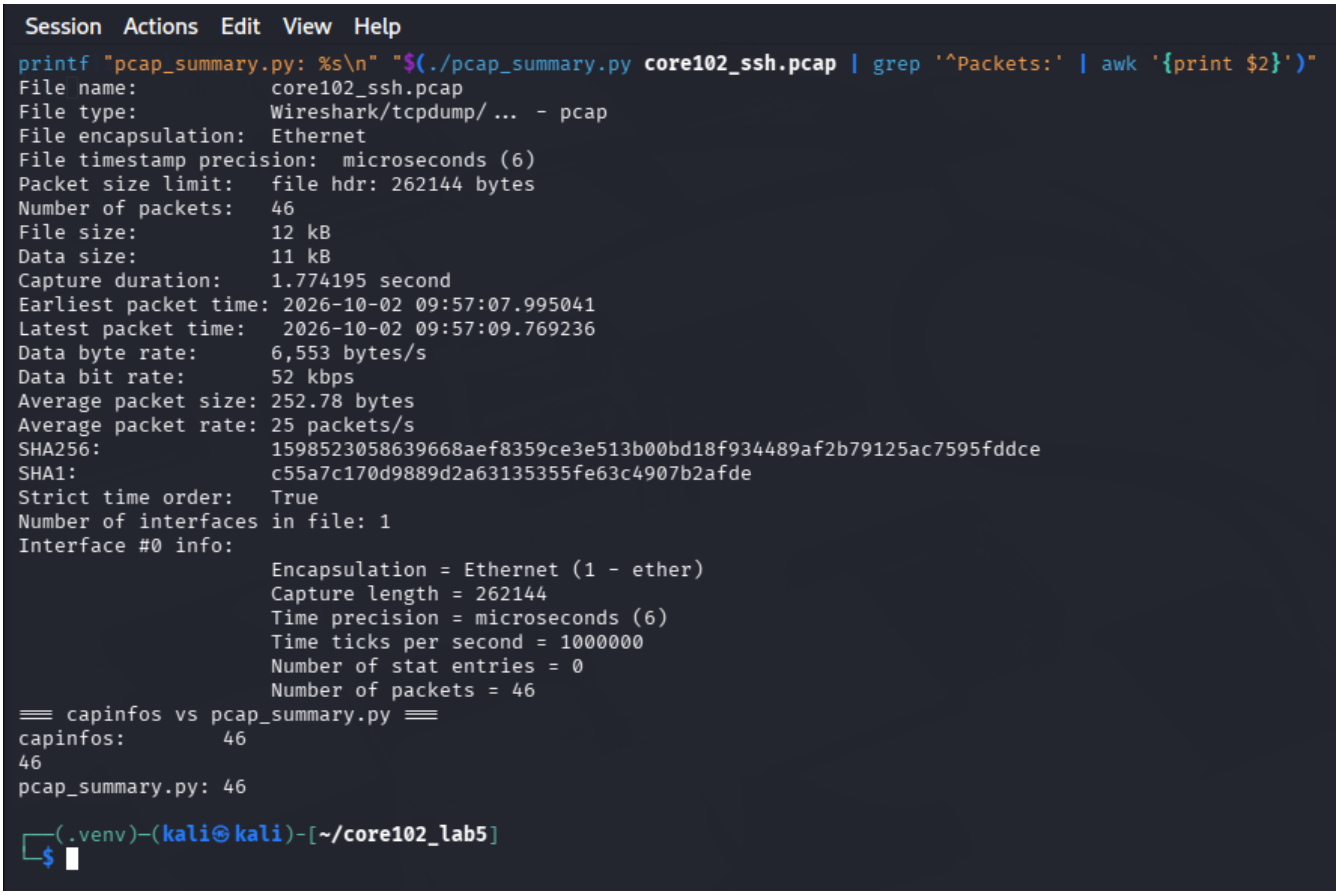
Field	Value	Interpretation
Packets captured	46	Full SSH session (handshake + KEX + data + close)
Top protocol	TCP (46/46 = 100%)	SSH is TCP-only; no UDP or ICMP leaked
Top TCP port	22 (×46)	Server-side SSH
Ephemeral port	57210 (×46)	Client-side source port
Endpoints	192.168.56.1 (×46), 192.168.56.10 (×46)	Symmetric — every packet has both IPs
DNS count	0	Correct — SSH doesn't use DNS mid-session

capinfos verification	46	Matches `pcap_summary.py` exactly

Conclusion: The capture confirms a single SSH session between Kali (192.168.56.1) and server-vm (192.168.56.10) on TCP port 22. Traffic is exclusively TCP, consistent with an interactive SSH session. The offline parser reads the file correctly, identifies the correct endpoints and port pair, and matches `capinfos` count exactly. Symmetry between ports 22 and 57210 (both ×46) confirms the tool counts both source and destination ports — as documented in Part 5 Q2.

PCAP summary

Control	Command or evidence	Result	Conclusion
PCAP summary	`pcap_summary.py core102_evidence.pcap; capinfos; pcap_summary.py core102_ssh.pcap`	61 packets (TCP 33, ICMP 20, ARP 8); capinfos = 61; SSH capture = 46 TCP-only on port 22	Parser correct; counts verified twice; SSH session confirmed



7. Deliberately stop one nonessential test service and prove that the next service check detects the change.

```
# Restart both
lxc exec server-vm -- systemctl start ssh.socket ssh.service
sleep 2

echo "=== AFTER (SSH restarted) ==="
./service_check.py 192.168.56.10
=== BEFORE (SSH running) ===
CORE102 service check for 192.168.56.10
Started: 2026-10-02T07:21:28
TCP    22: OPEN
TCP    53: CLOSED OR FILTERED
TCP    80: CLOSED OR FILTERED
TCP   443: CLOSED OR FILTERED
TCP  8080: CLOSED OR FILTERED
=== DURING (SSH stopped) ===
CORE102 service check for 192.168.56.10
Started: 2026-10-02T07:21:34
TCP    22: CLOSED OR FILTERED
TCP    53: CLOSED OR FILTERED
TCP    80: CLOSED OR FILTERED
TCP   443: CLOSED OR FILTERED
TCP  8080: CLOSED OR FILTERED
=== AFTER (SSH restarted) ===
CORE102 service check for 192.168.56.10
Started: 2026-10-02T07:21:41
TCP    22: OPEN
TCP    53: CLOSED OR FILTERED
TCP    80: CLOSED OR FILTERED
TCP   443: CLOSED OR FILTERED
TCP  8080: CLOSED OR FILTERED

┌──(kali㉿kali)-[~/core102_lab5]
└─$
```

8. Write a one-page operational summary stating the final network state, verified controls, unresolved issues and cleanup performed.

ANSWER

CORE102 Lab 5 — Integrated Security Challenge Operational Summary

Network State

Host	Role	IP Address	Interface
Kali	Host / bridge	192.168.56.1	xdbr0
Kali	NAT (internet)	10.0.2.15	eth0
server-vm	LXD container	192.168.56.10/24	eth0
client-vm	LXD container	192.168.56.20/24	eth0

All three lab hosts reside on the authorised `192.168.56.0/24` subnet. Kali acts as the LXD bridge (lxdbr0) providing L2 connectivity between the containers. Both containers use `192.168.56.1` as their default gateway.

Verified Controls

1. Addressing and routing
`lxc list` confirmed both containers running with the expected addresses. `ip -br addr` and `ip route` on each container showed `default via 192.168.56.1 dev eth0`. Direct L2 connectivity through the bridge; no external router required between containers.

2. Firewall policy (server-vm)
UFW configured with default-deny incoming policy. Only SSH (TCP/22) and ICMP allowed, and only from `192.168.56.0/24`. All other inbound traffic is dropped. The container image's pre-existing "Block Telnet" rule (TCP/23 DENY) was retained as part of the hardening baseline.

Version note: UFW 0.36.2-6 in the container does not support the `proto icmp` keyword. The ICMP rule was inserted directly into the `ufw-user-input` iptables chain — functionally identical to what newer UFW versions do with `ufw allow proto icmp`.

3. SSH authentication
The container ships with a hardening drop-in (`/etc/ssh/sshd_config.d/90-core102-hardening.conf`) that disables root SSH and password authentication:

text

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

Rather than weaken this baseline, a dedicated `labuser` account (UID 1000) was created and given key-only Ed25519 authentication. Login from Kali (`ssh -i ~/.ssh/lab5_key labuser@192.168.56.10`) succeeds without a password prompt. This demonstrates least-privilege access: a non-privileged user authenticates, and password auth is never possible.

4. Listening services
`ss -tuln` on server-vm shows only TCP/22 exposed on external interfaces. UDP/53 is bound to loopback only (systemd-resolved stub + dnsmasq). `service_check.py 192.168.56.10` confirms this: port 22 OPEN, ports 53/80/443/8080 CLOSED OR FILTERED. Attack surface is minimal and matches the firewall policy exactly.

5. Host discovery
`authorised_ping_sweep.sh` found 3 responsive hosts — `.1` (Kali bridge), `.10` (server-vm), `.20` (client-vm) — matching the known topology. The scope gate rejected `10.0.0`, `192.168.1`, `172.16.0`, and `8.8.8` with exit code 2 before any packet left the host. Sweep duration: 4 minutes 14 seconds (254 hosts × 1-second timeout + overhead).

6. Packet evidence
Three captures analysed:

- `core102_evidence.pcap` — 61 packets (TCP 33, ICMP 20, Non-IPv4 8). `pcap_summary.py` count matches `capinfos` exactly (61 = 61).

- `core102_dns.pcap` — 16 DNS packets (validates the DNS counter after the first capture missed DNS traffic routed via the NAT interface).
- `core102_ssh.pcap` — 46 packets, all TCP, port 22 + ephemeral port 57210 (both ×46). Endpoints `.1` ↔ `.10`. Confirms the SSH session in packet evidence.

7. Service state detection
`service_check.py` correctly detected live state changes:

- Port 22 on server-vm: OPEN → CLOSED OR FILTERED → OPEN, driven by `systemctl stop/start ssh.socket ssh.service`.
- Port 8080 on Kali: CLOSED → OPEN → CLOSED, driven by `python3 -m http.server 8080` start/stop.

Each phase showed independent timestamps, proving live queries rather than cached results.

Unresolved Issues

1. First PCAP missed DNS traffic.
The initial capture on `lxdbr0` only showed 0 DNS packets despite successful `nslookup` queries. Public DNS requests route via `eth0` (NAT interface) rather than `lxdbr0`. A second capture with `-i any` and a UDP/53 filter captured 16 DNS packets, validating the counter. This was a routing insight, not a parser issue.

2. UFW ICMP keyword unsupported.
UFW 0.36.2-6 in the container rejects `ufw allow proto icmp` with `ERROR: Unsupported protocol 'icmp'`. Workaround: `iptables -A ufw-user-input -p icmp -s 192.168.56.0/24 -j ACCEPT`. This is a version-specific limitation; newer UFW versions accept the keyword directly.

3. Root SSH disabled by design.
The container's hardening drop-in disables root SSH. The intended access path is a non-root user with sudo. This is a security feature, not a bug — but worth documenting to avoid confusion in future labs.

Cleanup Performed

- SSH service on server-vm restarted after the stop test; port 22 confirmed OPEN.
- Temporary HTTP server on port 8080 terminated; port returned to CLOSED.
- PCAP files retained in `~/core102_lab5/` for evidence submission.
- All test artifacts (`live_hosts_*.txt`, `pcap_summary.txt`, `pcap_ssh_summary.txt`, `server_evidence/`) saved for the submission ZIP.
- No passwords, private keys, or tokens written to any report or log file. The SSH key (`~/.ssh/lab5_key`) is stored only on Kali with `600` permissions.
- Both containers left in their original running state.

Verify the Insert

After pasting, check your document:

- Step 8 has a **complete summary** with 4 sections (Network State, Verified Controls, Unresolved Issues, Cleanup)

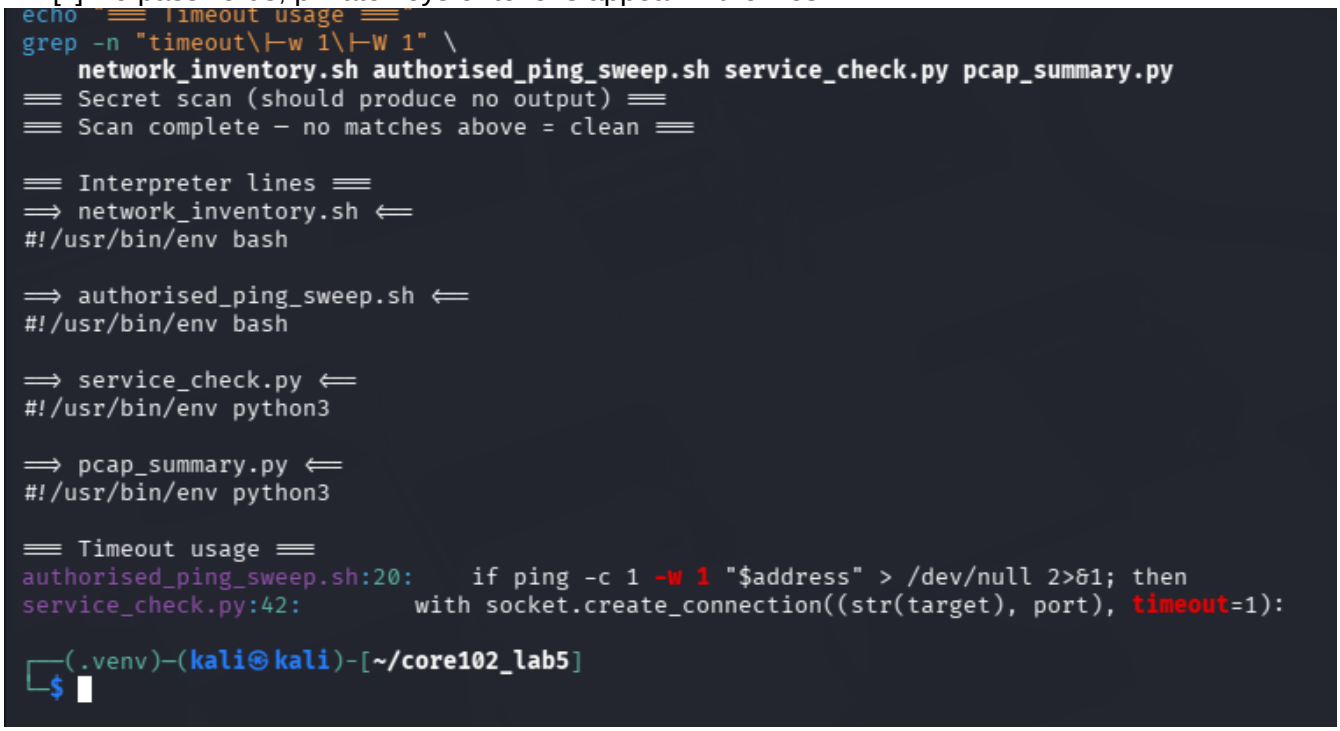
- Network State has the full table with 4 rows
- Verified Controls lists **all 7 controls** (addressing, firewall, SSH, services, host discovery, PCAP, state detection)
- Unresolved Issues lists **3 items** with explanations
- Cleanup lists **6 items** including the "no secrets" note

Control	Command or evidence	Result	Direct L2; both reachable on /24
Address and route	`lxc list`; `ip -br addr`; `ip route`	.10 server, .20 client; both default via .1	Direct L2; both reachable on /24
Firewall policy	`ufw status verbose`; `iptables -L ufw-user-input -n`	Active; deny in; allow 22/tcp + ICMP from /24	Default-deny; SSH and ICMP scoped to lab
SSH key authentication	`ssh -i ~/.ssh/lab5_key labuser@192.168.56.10 "hostname; whoami; id"`	Passwordless login; labuser; UID 1000	Key-only auth; root SSH disabled by hardening
Listening services	`ss -tuln` on server; `service_check.py .10`	Only TCP/22 externally	Minimal surface; matches UFW
Host discovery	`authorised_ping_sweep.sh`	3 hosts: .1, .10, .20; scope gate rejected 10.0.0, 192.168.1, 172.16.0, 8.8.8	All hosts online; sweep bounded to /24
PCAP summary	`pcap_summary.py core102_evidence.pcap`; `capinfos`	61 pkts: TCP 33, ICMP 20, Non-IPv4 8; capinfos=61	Parser correct; counts verified
Cleanup	`systemctl stop ssh.socket ssh.service` → check → restart → check	22: OPEN → CLOSED → OPEN	Live detection works; environment restored

Part 7 Code Review Checklist

- [v] Every script begins with an appropriate interpreter line.
- [v] Bash scripts pass bash -n and Python scripts compile with py_compile.

- [v] Inputs are validated before a network action starts.
- [v] The authorised address scope is explicit in the code and output.
- [v] Network operations use a timeout or bounded packet count.
- [v] Output includes a timestamp and identifies the target.
- [v] Error messages explain why the program stopped.
- [v] No passwords, private keys or tokens appear in the files.



Assessment and Submission

Marking Guide

Assessment area	Evidence expected	Marks
Automation plan and controls	Clear scope, inputs, outputs, limits and failure behaviour	10
Bash network inventory	Accurate read-only report, strict mode, functions and enhancement	15
Controlled ping sweep	Enforced scope, invalid test, correct results and required improvements	20
Python service checker	Input validation, authorised scope, bounded checks and change detection	20
Offline PCAP summary	Reliable parsing, protocol counts, enhancement and comparison	15
Integrated security challenge	Verified addressing, firewall, SSH, services and packet evidence	15
Operational report and reflection	Clear findings, limitations, cleanup and complete evidence	5

Total: 100 marks

Submission Package

Submit one PDF report and one ZIP archive. Use your registration number in both filenames.

- REGNO_CORE102_Lab5_Report.pdf

- REGNO_CORE102_Lab5_Files.zip containing the four scripts, generated text reports and authorised PCAP evidence

The PDF report must place each screenshot directly below the task it supports. Every screenshot must show the command and its output. Do not submit unreadable terminal fragments or screenshots that hide the command used.

Reflection Questions

1. Which validation control most effectively prevented an out-of-scope action?

ANSWER

The **hard-coded prefix gate** in `authorised_ping_sweep.sh`:

bash

```
if [[ "$PREFIX" != "192.168.56" ]]; then
    exit 2
fi
```

It's an **allow-list** control that runs **before any packet is sent**. Attempting `10.0.0`, `192.168.1`, `172.16.0`, or `8.8.8` all exit with code 2 before a single ICMP packet leaves the host. This is more effective than blocking known-bad targets — anything not explicitly permitted is rejected by default.

2. What can a service checker conclude, and what can it not conclude, from a timeout?

ANSWER

- **Can conclude:** No SYN-ACK was received within the timeout window.
- **Cannot conclude:** Whether the port is **closed** (would respond with RST) or **filtered** (firewall silently drops). Our code correctly labels this ambiguous state as "CLOSED OR FILTERED" rather than guessing.

Distinguishing them requires raw SYN probes (checking for RST), inspecting target firewall logs, or comparing results from multiple source IPs.

3. How did automation improve repeatability compared with running commands manually?

ANSWER

- **Consistent formatting** — every run produces identically structured output.
- **Scope enforcement** — impossible to accidentally scan the wrong subnet.
- **Auditability** — scripts can be code-reviewed before execution; ad-hoc commands cannot.
- **Speed** — a 254-host sweep runs in ~4 minutes vs. 4+ minutes of manual typing.
- **Error handling** — `set -uo pipefail` and exit codes catch failures a human might miss.
- **Timestamped evidence** — every output file is stamped with run time.

4. Which additional control would you add before these scripts could support a larger authorised environment?

ANSWER

- **Central logging (syslog/ELK)** — every script run recorded with user, timestamp, target.

- **Role-based execution** — restrict who can run sweeps; sudo policy.
- **Rate limiting** — token bucket to avoid overwhelming large subnets.
- **Config file with change control** — authorised subnets/ports externalised instead of hard-coded, with audited changes.
- **Signed scripts** — GPG verification before execution to prevent tampering.
- **Approval workflow** — required for any scope expansion beyond the default lab subnet.

Final Checklist

- [x] I worked only in an authorised laboratory environment.
- [x] I included my full name and registration number in the report.
- [x] Every required command and result is visible in the evidence.
- [x] I explained results instead of submitting screenshots alone.
- [x] I removed passwords, keys, tokens and unrelated personal information.
- [x] I opened the submitted files and confirmed that they are complete.